Chapter 6 Files and Exceptions

Topics

6.1 Introduction to File Input and Output

Concept:

When a program needs to save data for later use, it writes the data in a file. The data can be read from the file at a later time.

The programs you have written so far require the user to reenter data each time the program runs, because data stored in RAM (referenced by variables) disappears once the program stops running. If a program is to retain data between the times it runs, it must have a way of saving it. Data is saved in a file, which is usually stored on a computer's disk. Once the data is saved in a file, it will remain there after the program stops running. Data stored in a file can be retrieved and used at a later time.

Most of the commercial software packages that you use on a day-to-day basis store data in files. The following are a few examples:

- **Word processors.** Word processing programs are used to write letters, memos, reports, and other documents. The documents are

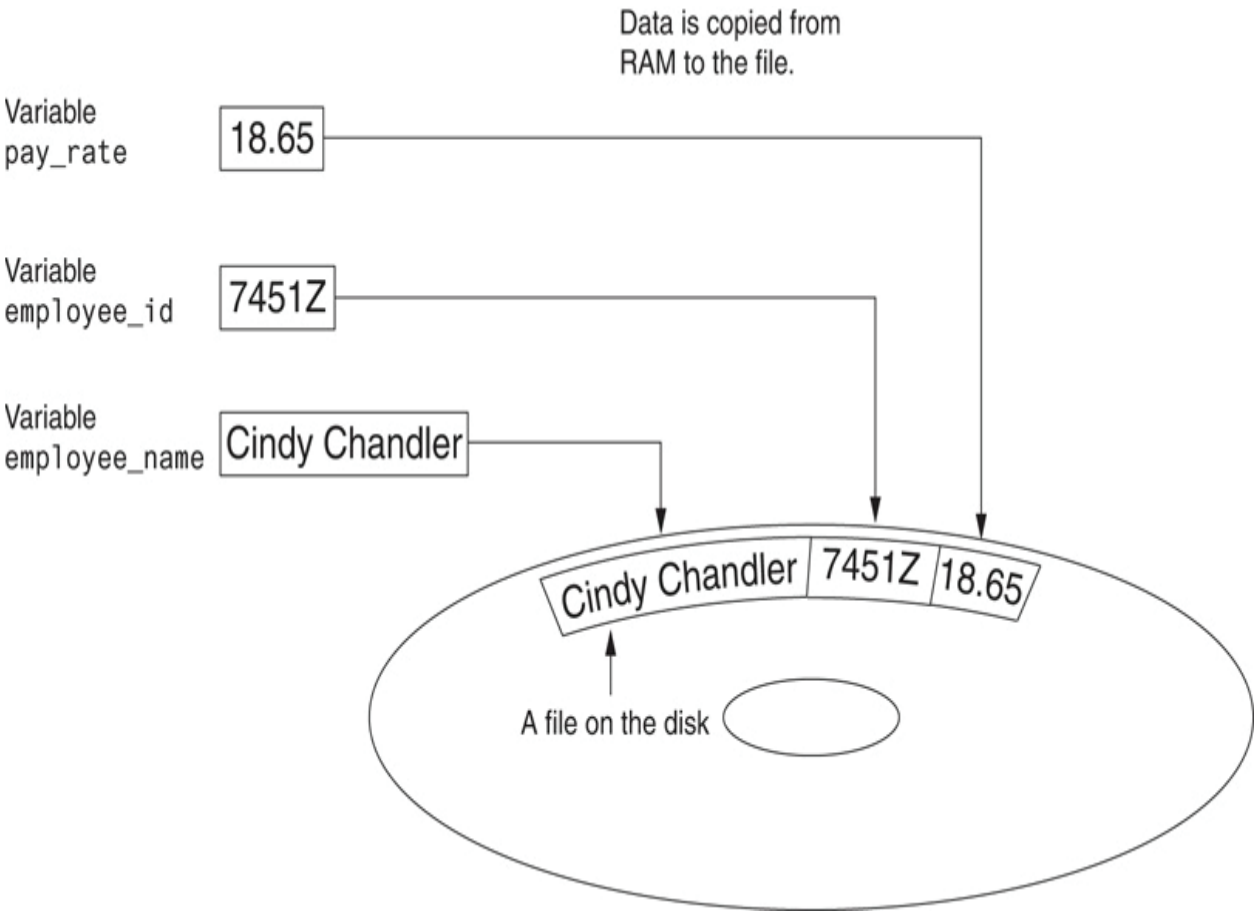
then saved in files so they can be edited and printed.

- **Image editors.** Image editing programs are used to draw graphics and edit images, such as the ones that you take with a digital camera. The images that you create or edit with an image editor are saved in files.
- **Spreadsheets.** Spreadsheet programs are used to work with numerical data. Numbers and mathematical formulas can be inserted into the rows and columns of the spreadsheet. The spreadsheet can then be saved in a file for use later.
- **Games.** Many computer games keep data stored in files. For example, some games keep a list of player names with their scores stored in a file. These games typically display the players' names in order of their scores, from highest to lowest. Some games also allow you to save your current game status in a file so you can quit the game and then resume playing it later without having to start from the beginning.
- **Web browers.** Sometimes when you visit a Web page, the browser stores a small file known as a *cookie* on your computer. Cookies typically contain information about the browsing session, such as the contents of a shopping cart.

Programs that are used in daily business operations rely extensively on files. Payroll programs keep employee data in files, inventory programs keep data about a company's products in files, accounting systems keep data about a company's financial operations in files, and so on.

Programmers usually refer to the process of saving data in a file as “writing data” to the file. When a piece of data is written to a file, it is copied from a variable in RAM to the file. This is illustrated in **Figure 6-1**. The term *output file* is used to describe a file that data is written to. It is called an output file because the program stores output in it.

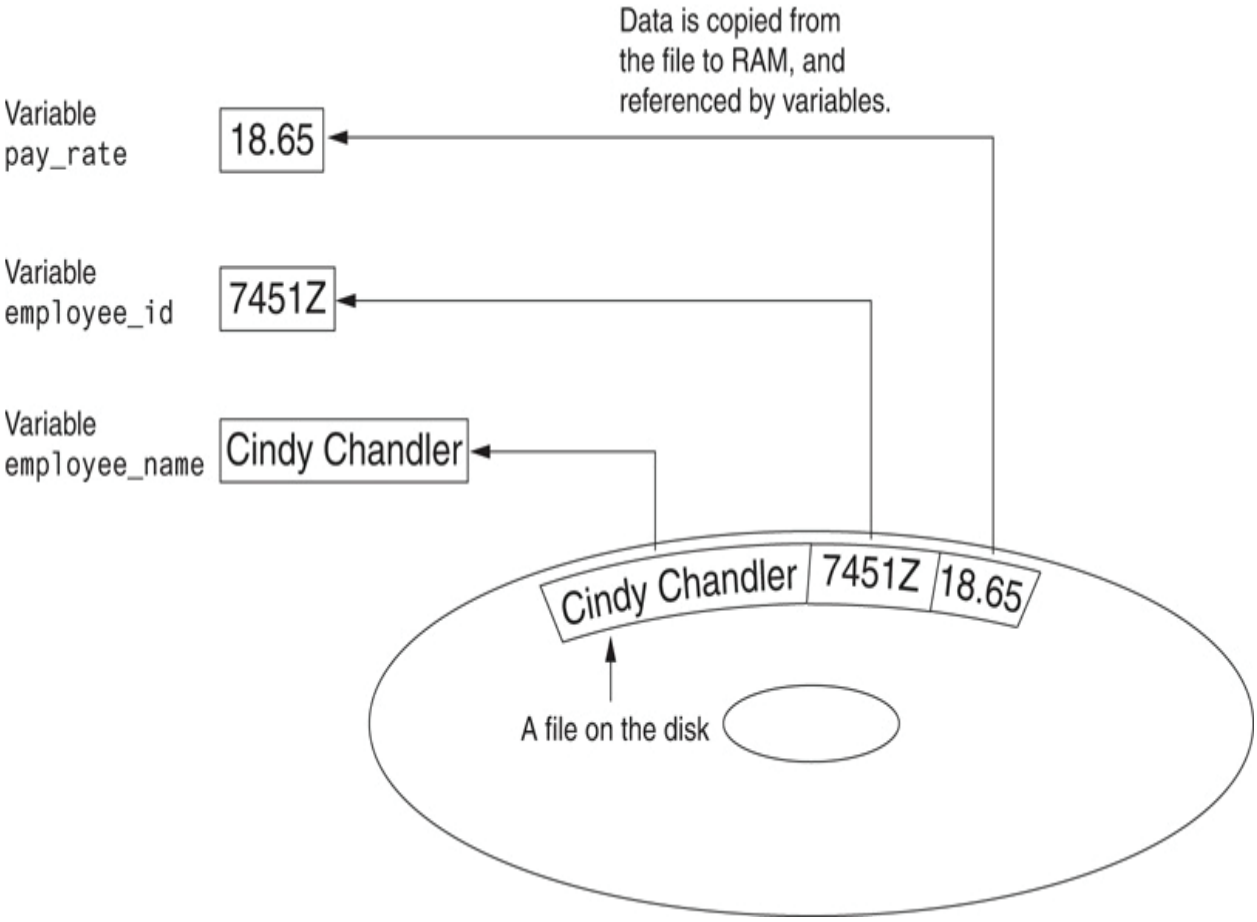
Figure 6-1 Writing data to a file



The process of retrieving data from a file is known as “reading data” from the file. When a piece of data is read from a file, it is copied from the file into RAM and referenced by a variable. **Figure 6-2** illustrates this. The term *input file* is used to describe a file from which

data is read. It is called an input file because the program gets input from the file.

Figure 6-2 Reading data from a file



This chapter discusses how to write data to files and read data from files. There are always three steps that must be taken when a file is used by a program.

1. **Open the file.** Opening a file creates a connection between the file and the program. Opening an output file usually creates the file on the disk and allows the program to write data to it.

Opening an input file allows the program to read data from the file.

2. **Process the file.** In this step, data is either written to the file (if it is an output file) or read from the file (if it is an input file).
3. **Close the file.** When the program is finished using the file, the file must be closed. Closing a file disconnects the file from the program.

Types of Files

In general, there are two types of files: text and binary. A *text file* contains data that has been encoded as text, using a scheme such as ASCII or Unicode. Even if the file contains numbers, those numbers are stored in the file as a series of characters. As a result, the file may be opened and viewed in a text editor such as Notepad. A *binary file* contains data that has not been converted to text. The data that is stored in a binary file is intended only for a program to read. As a consequence, you cannot view the contents of a binary file with a text editor.

Although Python allows you to work both text files and binary files, we will work only with text files in this book. That way, you will be able to use an editor to inspect the files that your programs create.

File Access Methods

Most programming languages provide two different ways to access data stored in a file: sequential access and direct access. When you work with a *sequential access file*, you access data from the beginning of the file to the end of the file. If you want to read a piece of data that is stored at the very end of the file, you have to read all of the data that comes before it—you cannot jump directly to the desired data. This is similar to the way older cassette tape players work. If you want to listen to the last song on a cassette tape, you have to either fast-forward over all of the songs that come before it or listen to them. There is no way to jump directly to a specific song.

When you work with a *direct access file* (which is also known as a *random access file*), you can jump directly to any piece of data in the file without reading the data that comes before it. This is similar to the way a CD player or an MP3 player works. You can jump directly to any song that you want to listen to.

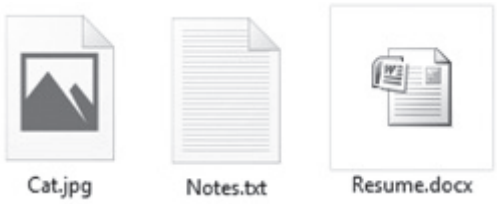
In this book, we will use sequential access files. Sequential access files are easy to work with, and you can use them to gain an understanding of basic file operations.

Filenames and File Objects

Most computer users are accustomed to the fact that files are identified by a filename. For example, when you create a document with a word processor and save the document in a file, you have to specify a filename. When you use a utility such as Windows Explorer

to examine the contents of your disk, you see a list of filenames. **Figure 6-3** shows how three files named `cat.jpg`, `notes.txt`, and `resume.docx` might be graphically represented in Windows.

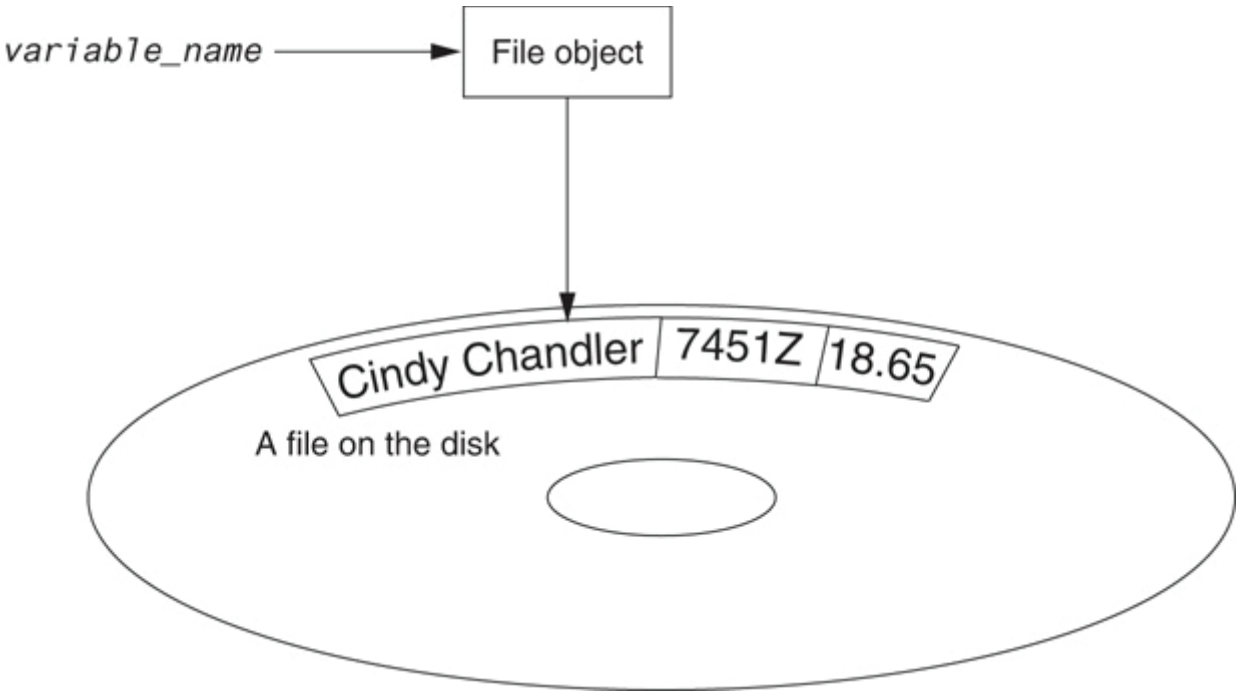
Figure 6-3 Three files



Each operating system has its own rules for naming files. Many systems support the use of *filename extensions*, which are short sequences of characters that appear at the end of a filename preceded by a period (which is known as a “dot”). For example, the files depicted in **Figure 6-3** have the extensions `.jpg`, `.txt`, and `.doc`. The extension usually indicates the type of data stored in the file. For example, the `.jpg` extension usually indicates that the file contains a graphic image that is compressed according to the JPEG image standard. The `.txt` extension usually indicates that the file contains text. The `.doc` extension (as well as the `.docx` extension) usually indicates that the file contains a Microsoft Word document.

In order for a program to work with a file on the computer’s disk, the program must create a file object in memory. A *file object* is an object that is associated with a specific file and provides a way for the program to work with that file. In the program, a variable references the file object. This variable is used to carry out any operations that are performed on the file. This concept is shown in **Figure 6-4**.

Figure 6-4 A variable name references a file object that is associated with a file



Opening a File

You use the `open` function in Python to open a file. The `open` function creates a file object and associates it with a file on the disk. Here is the general format of how the `open` function is used:

```
file_variable = open(filename, mode)
```

In the general format:

- *file_variable* is the name of the variable that will reference the file object.
- *filename* is a string specifying the name of the file.
- *mode* is a string specifying the mode (reading, writing, etc.) in which the file will be opened. **Table 6-1** shows three of the strings that you can use to specify a mode. (There are other, more complex modes. The modes shown in **Table 6-1** are the ones we will use in this book.)

Table 6-1 Some of the Python file modes

Mode	Description
`'r'`	Open a file for reading only. The file cannot be changed or written to.
`'w'`	Open a file for writing. If the file already exists, erase its contents. If it does not exist, create it.
`'a'`	Open a file to be written to. All data written to the file will be appended to its end. If the file does not exist, create it.

For example, suppose the file `customers.txt` contains customer data, and we want to open it for reading. Here is an example of how we would call the `open` function:

```
customer_file = open('customers.txt', 'r')
```

After this statement executes, the file named `customers.txt` will be opened, and the variable `customer_file` will reference a file object that we can use to read data from the file.

Suppose we want to create a file named `sales.txt` and write data to it. Here is an example of how we would call the `open` function:

```
sales_file = open('sales.txt', 'w')
```

After this statement executes, the file named `sales.txt` will be created, and the variable `sales_file` will reference a file object that we can use to write data to the file.

Warning:

Remember, when you use the `'w'` mode, you are creating the file on the disk. If a file with the specified name already exists when the file is opened, the contents of the existing file will be deleted.

Specifying the Location of a File

When you pass a file name that does not contain a path as an argument to the `open` function, the Python interpreter assumes the file's location is the same as that of the program. For example, suppose a program is located in the following folder on a Windows computer:

```
C:\Users\Blake\Documents\Python
```

If the program is running and it executes the following statement, the file `test.txt` is created in the same folder:

```
test_file = open('test.txt', 'w')
```

If you want to open a file in a different location, you can specify a path as well as a filename in the argument that you pass to the `open` function. If you specify a path in a string literal (particularly on a Windows computer), be sure to prefix the string with the letter `r`. Here is an example:

```
test_file = open(r'C:\Users\Blake\temp\test.txt', 'w')
```

This statement creates the file `test.txt` in the folder `C:\Users\Blake\temp`. The `r` prefix specifies that the string is a *raw string*. This causes the Python interpreter to read the backslash

characters as literal backslashes. Without the `r` prefix, the interpreter would assume that the backslash characters were part of escape sequences, and an error would occur.

Writing Data to a File

So far in this book, you have worked with several of Python's library functions, and you have even written your own functions. Now, we will introduce you to another type of function, which is known as a method. A *method* is a function that belongs to an object and performs some operation using that object. Once you have opened a file, you use the file object's methods to perform operations on the file.

For example, file objects have a method named `write` that can be used to write data to a file. Here is the general format of how you call the `write` method:

```
file_variable.write(string)
```

In the format, *`file_variable`* is a variable that references a file object, and *`string`* is a string that will be written to the file. The file must be opened for writing (using the `'w'` or `'a'` mode) or an error will occur.

Let's assume `customer_file` references a file object, and the file was opened for writing with the `'w'` mode. Here is an example of how we would write the string 'Charles Pace' to the file:

```
customer_file.write('Charles Pace')
```

The following code shows another example:

```
name = 'Charles Pace'
customer_file.write(name)
```

The second statement writes the value referenced by the `name` variable to the file associated with `customer_file`. In this case, it would write the string 'Charles Pace' to the file. (These examples show a string being written to a file, but you can also write numeric values.)

Once a program is finished working with a file, it should close the file. Closing a file disconnects the program from the file. In some systems, failure to close an output file can cause a loss of data. This happens because the data that is written to a file is first written to a *buffer*, which is a small "holding section" in memory. When the buffer is full, the system writes the buffer's contents to the file. This technique increases the system's performance, because writing data to memory is faster than writing it to a disk. The process of closing an output file

forces any unsaved data that remains in the buffer to be written to the file.

In Python, you use the file object's `close` method to close a file. For example, the following statement closes the file that is associated with `customer_file`:

```
customer_file.close()
```

Program 6-1 shows a complete Python program that opens an output file, writes data to it, then closes it.

Program 6-1 *(file_write.py)*

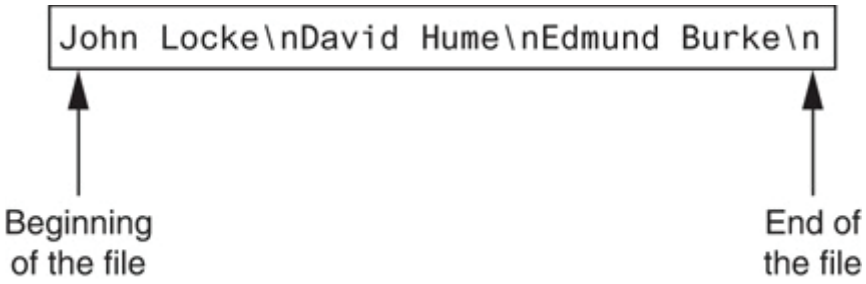
```
1  # This program writes three lines of data
2  # to a file.
3  def main():
4      # Open a file named philosophers.txt.
5      outfile = open('philosophers.txt', 'w')
6
7      # Write the names of three philosphers
8      # to the file.
9      outfile.write('John Locke\n')
10     outfile.write('David Hume\n')
11     outfile.write('Edmund Burke\n')
12
```

```
13      # Close the file.
14      outfile.close()
15
16  # Call the main function.
17  main()
```

Line 5 opens the file `philosophers.txt` using the `'w'` mode. (This causes the file to be created and opens it for writing.) It also creates a file object in memory and assigns that object to the `outfile` variable.

The statements in lines 9 through 11 write three strings to the file. Line 9 writes the string `'John Locke\n'`, line 10 writes the string `'David Hume\n'`, and line 11 writes the string `'Edmund Burke\n'`. Line 14 closes the file. After this program runs, the three items shown in **Figure 6-5** will be written to the `philosophers.txt` file.

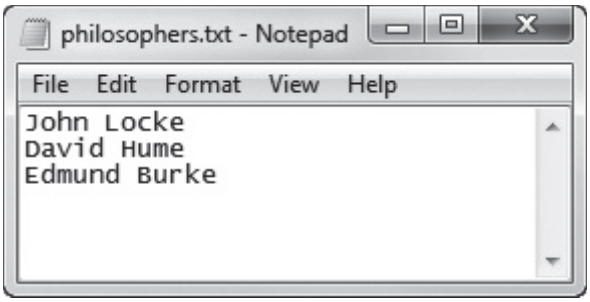
Figure 6-5 Contents of the file `philosophers.txt`



Notice each of the strings written to the file end with `\n`, which you will recall is the newline escape sequence. The `\n` not only separates the items that are in the file, but also causes each of them to appear in a

separate line when viewed in a text editor. For example, **Figure 6-6** shows the `philosophers.txt` file as it appears in Notepad.

Figure 6-6 Contents of `philosophers.txt` in Notepad



Reading Data From a File

If a file has been opened for reading (using the `'r'` mode) you can use the file object's `read` method to read its entire contents into memory. When you call the `read` method, it returns the file's contents as a string. For example, **Program 6-2** shows how we can use the `read` method to read the contents of the `philosophers.txt` file we created earlier.

Program 6-2 (file_read.py)

```
1  # This program reads and displays the contents
2  # of the philosophers.txt file.
3  def main():
4      # Open a file named philosophers.txt.
```

```
 5      infile = open('philosophers.txt', 'r')
 6
 7      # Read the file's contents.
 8      file_contents = infile.read()
 9
10      # Close the file.
11      infile.close()
12
13      # Print the data that was read into
14      # memory.
15      print(file_contents)
16
17  # Call the main function.
18  main()
```

Program Output

```
 John Locke
David Hume
Edmund Burke
```

The statement in line 5 opens the `philosophers.txt` file for reading, using the `'r'` mode. It also creates a file object and assigns the object to the `infile` variable. Line 8 calls the `infile.read` method to read the file's contents. The file's contents are read into memory as a string and assigned to the `file_contents` variable. This is shown in **Figure 6-**

7. Then the statement in line 15 prints the string that is referenced by the variable.

Figure 6-7 The `file_contents` variable references the string that was read from the file

Although the `read` method allows you to easily read the entire contents of a file with one statement, many programs need to read and process the items that are stored in a file one at a time. For example, suppose a file contains a series of sales amounts, and you need to write a program that calculates the total of the amounts in the file. The program would read each sale amount from the file and add it to an accumulator.

In Python, you can use the `readline` method to read a line from a file. (A line is simply a string of characters that are terminated with a `\n`.) The method returns the line as a string, including the `\n`. **Program 6-3** shows how we can use the `readline` method to read the contents of the `philosophers.txt` file, one line at a time.

Program 6-3 (line_read.py)

```
1  # This program reads the contents of the
2  # philosophers.txt file one line at a time.
3  def main():
4      # Open a file named philosophers.txt.
```

```
 5      infile = open('philosophers.txt', 'r')
 6
 7      # Read three lines from the file.
 8      line1 = infile.readline()
 9      line2 = infile.readline()
10      line3 = infile.readline()
11
12      # Close the file.
13      infile.close()
14
15      # Print the data that was read into
16      # memory.
17      print(line1)
18      print(line2)
19      print(line3)
20
21  # Call the main function.
22  main()
```

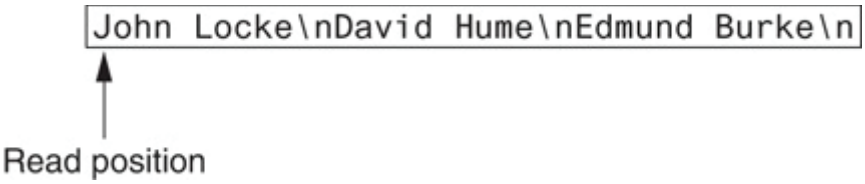
Program Output

```
John Locke

David Hume

Edmund Burke
```

Before we examine the code, notice that a blank line is displayed after each line in the output. This is because each item that is read from the file ends with a newline character (`\n`). Later, you will learn how to remove the newline character.

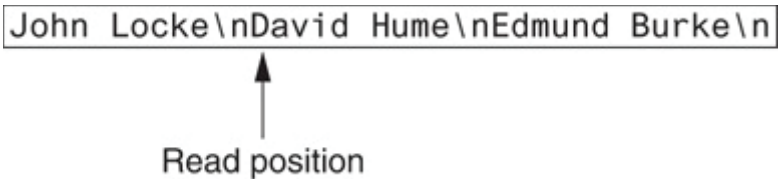
The statement in line 5 opens the `philosophers.txt` file for reading, using the `'r'` mode. It also creates a file object and assigns the object to the `infile` variable. When a file is opened for reading, a special value known as a *read position* is internally maintained for that file. A file's read position marks the location of the next item that will be read from the file. Initially, the read position is set to the beginning of the file. After the statement in line 5 executes, the read position for the `philosophers.txt` file will be positioned as shown in **Figure 6-8**.

Figure 6-8 Initial read position



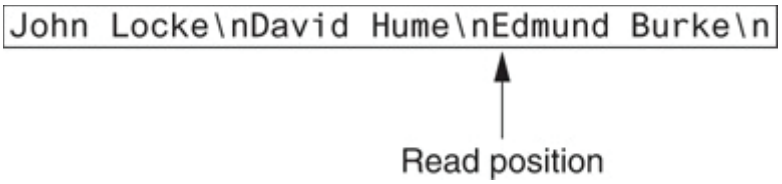
The statement in line 8 calls the `infile.readline` method to read the first line from the file. The line, which is returned as a string, is assigned to the `line1` variable. After this statement executes the `line1` variable will be assigned the string `'John Locke\n'`. In addition, the file's read position will be advanced to the next line in the file, as shown in **Figure 6-9**.

Figure 6-9 Read position advanced to the next line



Then the statement in line 9 reads the next line from the file and assigns it to the `line2` variable. After this statement executes the `line2` variable will reference the string `'David Hume\n'`. The file's read position will be advanced to the next line in the file, as shown in **Figure 6-10**.

Figure 6-10 Read position advanced to the next line



Then the statement in line 10 reads the next line from the file and assigns it to the `line3` variable. After this statement executes, the `line3` variable will reference the string `'Edmund Burke\n'`. After this statement executes, the read position will be advanced to the end of the file, as shown in **Figure 6-11**. **Figure 6-12** shows the `line1`, `line2`, and `line3` variables and the strings they reference after these statements have executed.

Figure 6-11 Read position advanced to the end of the file

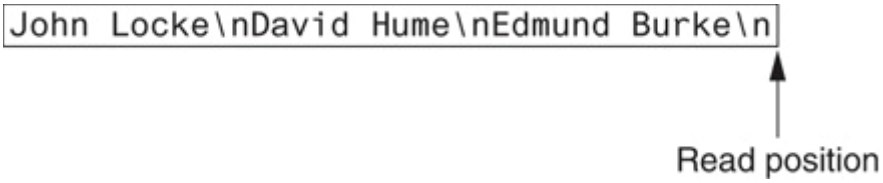
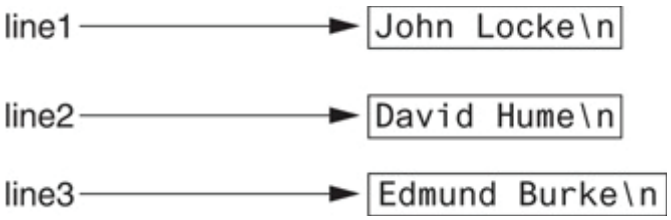


Figure 6-12 The strings referenced by the `line1`, `line2`, and `line3` variables



The statement in line 13 closes the file. The statements in lines 17 through 19 display the contents of the `line1`, `line2`, and `line3` variables.

> ***Note:***
>
> If the last line in a file is not terminated with a `\n`, the `readline` method will return the line without a `\n`.

Concatenating a Newline to a String

Program 6-1 wrote three string literals to a file, and each string literal ended with a `\n` escape sequence. In most cases, the data items that are written to a file are not string literals, but values in

memory that are referenced by variables. This would be the case in a program that prompts the user to enter data and then writes that data to a file.

When a program writes data that has been entered by the user to a file, it is usually necessary to concatenate a `\n` escape sequence to the data before writing it. This ensures that each piece of data is written to a separate line in the file. **Program 6-4** demonstrates how this is done.

Program 6-4 *(write_names.py)*

```
1  # This program gets three names from the user
2  # and writes them to a file.
3
4  def main():
5      # Get three names.
6      print('Enter the names of three friends.')
7      name1 = input('Friend #1: ')
8      name2 = input('Friend #2: ')
9      name3 = input('Friend #3: ')
10
11     # Open a file named friends.txt.
12     myfile = open('friends.txt', 'w')
13
14     # Write the names to the file.
15     myfile.write(name1 + '\n')
16     myfile.write(name2 + '\n')
```

```
17      myfile.write(name3 + '\n')
18
19      # Close the file.
20      myfile.close()
21      print('The names were written to friends.txt.')
22
23  # Call the main function.
24  main()
```

Program Output (with input shown in bold)

```
Enter the names of three friends.
Friend #1: Joe [Enter]
Friend #2: Rose [Enter]
Friend #3: Geri [Enter]
The names were written to friends.txt.
```

Lines 7 through 9 prompt the user to enter three names, and those names are assigned to the variables `name1`, `name2`, and `name3`. Line 12 opens a file named `friends.txt` for writing. Then, lines 15 through 17 write the names entered by the user, each with `'\n'` concatenated to it. As a result, each name will have the `\n` escape sequence added to it when written to the file. **Figure 6-13** shows the contents of the file with the names entered by the user in the sample run.

Figure 6-13 The `friends.txt` file

```
Joe\nRose\nGeri\n
```

Reading a String and Stripping the Newline from It

Sometimes complications are caused by the `\n` that appears at the end of the strings that are returned from the `readline` method. For example, did you notice in the sample output of **Program 6-3** that a blank line is printed after each line of output? This is because each of the strings that are printed in lines 17 through 19 end with a `\n` escape sequence. When the strings are printed, the `\n` causes an extra blank line to appear.

The `\n` serves a necessary purpose inside a file: it separates the items that are stored in the file. However, in many cases, you want to remove the `\n` from a string after it is read from a file. Each string in Python has a method named `rstrip` that removes, or "strips," specific characters from the end of a string. (It is named `rstrip` because it strips characters from the right side of a string.) The following code shows an example of how the `rstrip` method can be used.

```
name = 'Joanne Manchester\n'
name = name.rstrip('\n')
```

The first statement assigns the string `'Joanne Manchester\n'` to the `name` variable. (Notice the string ends with the `\n` escape sequence.) The second statement calls the `name.rstrip('\n')` method. The method returns a copy of the `name` string without the trailing `\n`. This string is assigned back to the `name` variable. The result is that the trailing `\n` is stripped away from the `name` string.

Program 6-5 is another program that reads and displays the contents of the `philosophers.txt` file. This program uses the `rstrip` method to strip the `\n` from the strings that are read from the file before they are displayed on the screen. As a result, the extra blank lines do not appear in the output.

Program 6-5 (strip_newline.py)

```
1  # This program reads the contents of the
2  # philosophers.txt file one line at a time.
3  def main():
4      # Open a file named philosophers.txt.
5      infile = open('philosophers.txt', 'r')
6
7      # Read three lines from the file.
8      line1 = infile.readline()
9      line2 = infile.readline()
10     line3 = infile.readline()
11
12     # Strip the \n from each string.
```

```
13      line1 = line1.rstrip('\n')
14      line2 = line2.rstrip('\n')
15      line3 = line3.rstrip('\n')
16
17      # Close the file.
18      infile.close()
19
20      # Print the data that was read into
21      # memory.
22      print(line1)
23      print(line2)
24      print(line3)
25
26  # Call the main function.
27  main()
```

Program Output

```
 John Locke
David Hume
Edmund Burke
```

Appending Data to an Existing File

When you use the `'w'` mode to open an output file and a file with the specified filename already exists on the disk, the existing file will be deleted and a new empty file with the same name will be created. Sometimes you want to preserve an existing file and append new data to its current contents. Appending data to a file means writing new data to the end of the data that already exists in the file.

In Python, you can use the `'a'` mode to open an output file in *append mode*, which means the following.

- If the file already exists, it will not be erased. If the file does not exist, it will be created.
- When data is written to the file, it will be written at the end of the file's current contents.

For example, assume the file `friends.txt` contains the following names, each in a separate line:

```
Joe
Rose
Geri
```

The following code opens the file and appends additional data to its existing contents.

```
myfile = open('friends.txt', 'a')
```

```
myfile.write('Matt\n')
myfile.write('Chris\n')
myfile.write('Suze\n')
myfile.close()
```

After this program runs, the file `friends.txt` will contain the following data:

```
Joe
Rose
Geri
Matt
Chris
Suze
```

Writing and Reading Numeric Data

Strings can be written directly to a file with the `write` method, but numbers must be converted to strings before they can be written. Python has a built-in function named `str` that converts a value to a string. For example, assuming the variable `num` is assigned the value 99, the expression `str(num)` will return the string `'99'`.

Program 6-6 shows an example of how you can use the `str` function to convert a number to a string, and write the resulting string to a file.

Program 6-6 *(write_numbers.py)*

```
1  # This program demonstrates how numbers
2  # must be converted to strings before they
3  # are written to a text file.
4
5  def main():
6      # Open a file for writing.
7      outfile = open('numbers.txt', 'w')
8
9      # Get three numbers from the user.
10     num1 = int(input('Enter a number: '))
11     num2 = int(input('Enter another number: '))
12     num3 = int(input('Enter another number: '))
13
14     # Write the numbers to the file.
15     outfile.write(str(num1) + '\n')
16     outfile.write(str(num2) + '\n')
17     outfile.write(str(num3) + '\n')
18
19     # Close the file.
20     outfile.close()
21     print('Data written to numbers.txt')
22
```

```
23  # Call the main function.
24  main()
```

Program Output (with input shown in bold)

```
Enter a number: 22 Enter
Enter another number: 14 Enter
Enter another number: -99 Enter
Data written to numbers.txt
```

The statement in line 7 opens the file `numbers.txt` for writing. Then the statements in lines 10 through 12 prompt the user to enter three numbers, which are assigned to the variables `num1`, `num2`, and `num3`.

Take a closer look at the statement in line 15, which writes the value referenced by `num1` to the file:

```
outfile.write(str(num1) + '\n')
```

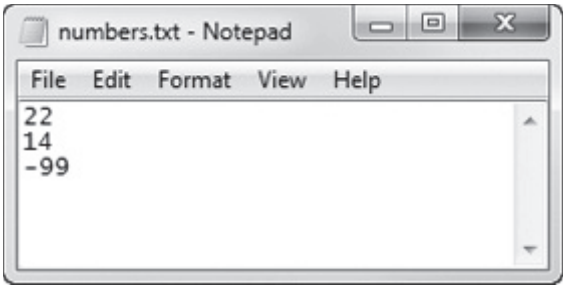
The expression `str(num1) + '\n'` converts the value referenced by `num1` to a string and concatenates the `\n` escape sequence to the string. In the program's sample run, the user entered 22 as the first number, so this expression produces the string `'22\n'`. As a result, the string `'22\n'` is written to the file.

Lines 16 and 17 perform the similar operations, writing the values referenced by `num2` and `num3` to the file. After these statements execute, the values shown in **Figure 6-14** will be written to the file. **Figure 6-15** shows the file viewed in Notepad.

Figure 6-14 Contents of the `numbers.txt` file

```
22\n14\n-99\n
```

Figure 6-15 The `numbers.txt` file viewed in Notepad



When you read numbers from a text file, they are always read as strings. For example, suppose a program uses the following code to read the first line from the `numbers.txt` file that was created by **Program 6-6**:

```
1  infile = open('numbers.txt', 'r')
2  value = infile.readline()
3  infile.close()
```

The statement in line 2 uses the `readline` method to read a line from the file. After this statement executes, the `value` variable will reference the string `'22\n'`. This can cause a problem if we intend to perform

math with the `value` variable, because you cannot perform math on strings. In such a case you must convert the string to a numeric type.

Recall from **Chapter 2** that Python provides the built-in function `int` to convert a string to an integer, and the built-in function `float` to convert a string to a floating-point number. For example, we could modify the code previously shown as follows:

```
1  infile = open('numbers.txt', 'r')
2  string_input = infile.readline()
3  value = int(string_input)
4  infile.close()
```

The statement in line 2 reads a line from the file and assigns it to the `string_input` variable. As a result, `string_input` will reference the string `'22\n'`. Then the statement in line 3 uses the `int` function to convert `string_input` to an integer, and assigns the result to `value`. After this statement executes, the `value` variable will reference the integer `22`. (Both the `int` and `float` functions ignore any `\n` at the end of the string that is passed as an argument.)

This code demonstrates the steps involved in reading a string from a file with the `readline` method then converting that string to an integer with the `int` function. In many situations, however, the code can be simplified. A better way is to read the string from the file and convert it in one statement, as shown here:

```
1  infile = open('numbers.txt', 'r')
2  value = int(infile.readline())
3  infile.close()
```

Notice in line 2 a call to the `readline` method is used as the argument to the `int` function. Here's how the code works: the `readline` method is called, and it returns a string. That string is passed to the `int` function, which converts it to an integer. The result is assigned to the `value` variable.

Program 6-7 shows a more complete demonstration. The contents of the `numbers.txt` file are read, converted to integers, and added together.

Program 6-7 (read_numbers.py)

```
1  # This program demonstrates how numbers that are
2  # read from a file must be converted from strings
3  # before they are used in a math operation.
4
5  def main():
6      # Open a file for reading.
7      infile = open('numbers.txt', 'r')
8
9      # Read three numbers from the file.
10     num1 = int(infile.readline())
```

```
11      num2 = int(infile.readline())
12      num3 = int(infile.readline())
13
14      # Close the file.
15      infile.close()
16
17      # Add the three numbers.
18      total = num1 + num2 + num3
19
20      # Display the numbers and their total.
21      print('The numbers are:', num1, num2, num3)
22      print('Their total is:', total)
23
24  # Call the main function.
25  main()
```

Program Output

```
The numbers are: 22 14 -99
Their total is: -63
```

Checkpoint

6.1 What is an output file?

6.2 What is an input file?

6.3 What three steps must be taken by a program when it uses a file?

6.4 In general, what are the two types of files? What is the difference between these two types of files?

6.5 What are the two types of file access? What is the difference between these two?

6.6 When writing a program that performs an operation on a file, what two file-associated names do you have to work with in your code?

6.7 If a file already exists, what happens to it if you try to open it as an output file (using the `'w'` mode)?

6.8 What is the purpose of opening a file?

6.9 What is the purpose of closing a file?

6.10 What is a file's read position? Initially, where is the read position when an input file is opened?

6.11 In what mode do you open a file if you want to write data to it, but you do not want to erase the file's existing contents? When you write data to such a file, to what part of the file is the data written?

6.2 Using Loops to Process Files

Concept:

Files usually hold large amounts of data, and programs typically use a loop to process the data in a file.

Using Loops to Process Files

Although some programs use files to store only small amounts of data, files are typically used to hold large collections of data. When a program uses a file to write or read a large amount of data, a loop is typically involved. For example, look at the code in **Program 6-8**. This program gets sales amounts for a series of days from the user and writes those amounts to a file named `sales.txt`. The user specifies the number of days of sales data he or she needs to enter. In

the sample run of the program, the user enters sales amounts for five days. **Figure 6-16** shows the contents of the `sales.txt` file containing the data entered by the user in the sample run.

Figure 6-16 Contents of the `sales.txt` file

```
1000.0\n2000.0\n3000.0\n4000.0\n5000.0\n
```

Program 6-8 (write_sales.py)

```
1  # This program prompts the user for sales amounts
2  # and writes those amounts to the sales.txt file.
3
4  def main():
5      # Get the number of days.
6      num_days = int(input('For how many days do ' +
7                           'you have sales? '))
8
9      # Open a new file named sales.txt.
10     sales_file = open('sales.txt', 'w')
11
12     # Get the amount of sales for each day and write
13     # it to the file.
14     for count in range(1, num_days + 1):
15         # Get the sales for a day.
16         sales = float(input('Enter the sales for day #' +
17                             str(count) + ': '))
18
19         # Write the sales amount to the file.
```

```
20          sales_file.write(str(sales) + '\n')
21
22      # Close the file.
23      sales_file.close()
24      print('Data written to sales.txt.')
25
26  # Call the main function.
27  main()
```

Program Output (with input shown in bold)

```
For how many days do you have sales? 5 Enter
Enter the sales for day #1: 1000.0 Enter
Enter the sales for day #2: 2000.0 Enter
Enter the sales for day #3: 3000.0 Enter
Enter the sales for day #4: 4000.0 Enter
Enter the sales for day #5: 5000.0 Enter
Data written to sales.txt.
```

Reading a File with a Loop and Detecting the End of the File

Quite often, a program must read the contents of a file without knowing the number of items that are stored in the file. For example,

the `sales.txt` file that was created by **Program 6-8** can have any number of items stored in it, because the program asks the user for the number of days for which he or she has sales amounts. If the user enters 5 as the number of days, the program gets 5 sales amounts and writes them to the file. If the user enters 100 as the number of days, the program gets 100 sales amounts and writes them to the file.

This presents a problem if you want to write a program that processes all of the items in the file, however many there are. For example, suppose you need to write a program that reads all of the amounts in the `sales.txt` file and calculates their total. You can use a loop to read the items in the file, but you need a way of knowing when the end of the file has been reached.

In Python, the `readline` method returns an empty string `('')` when it has attempted to read beyond the end of a file. This makes it possible to write a `while` loop that determines when the end of a file has been reached. Here is the general algorithm, in pseudocode:

Open the file

Use `readline` to read the first line from the file

While the value returned from `readline` is not an empty string:

    Process the item that was just read from the file

    Use `readline` to read the next line from the file.

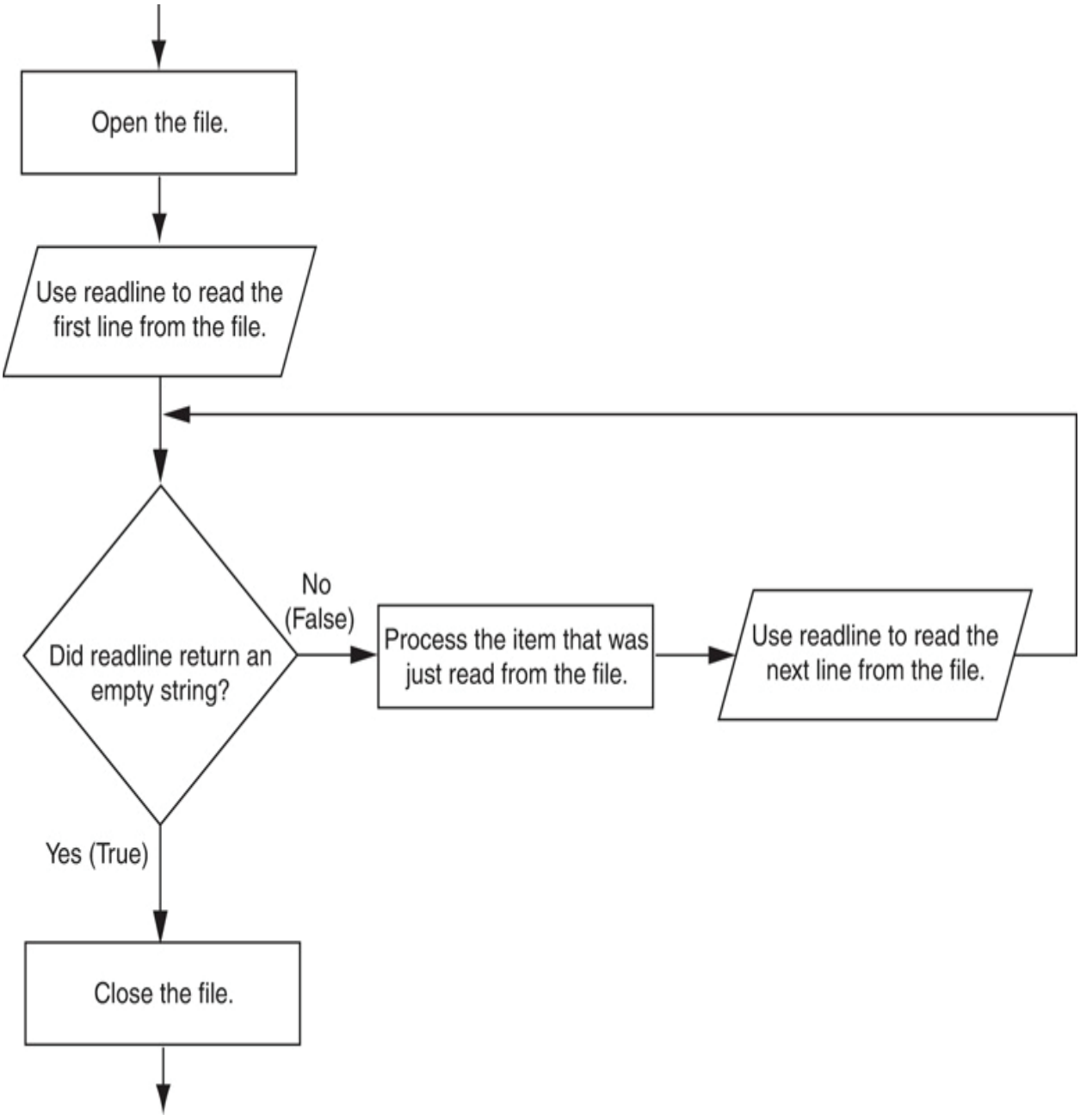
Close the file

Note:

In this algorithm, we call the `readline` method just before entering the `while` loop. The purpose of this method call is to get the first line in the file, so it can be tested by the loop. This initial read operation is called a *priming read*.

Figure 6-17 shows this algorithm in a flowchart.

Figure 6-17 General logic for detecting the end of a file



Program 6-9 demonstrates how this can be done in code. The program reads and displays all of the values in the `sales.txt` file.

Program 6-9 *(read_sales.py)*

```
1  # This program reads all of the values in
2  # the sales.txt file.
3
4  def main():
5      # Open the sales.txt file for reading.
6      sales_file = open('sales.txt', 'r')
7
8      # Read the first line from the file, but
9      # don't convert to a number yet. We still
10     # need to test for an empty string.
11     line = sales_file.readline()
12
13     # As long as an empty string is not returned
14     # from readline, continue processing.
15     while line != '':
16         # Convert line to a float.
17         amount = float(line)
18
19         # Format and display the amount.
20         print(format(amount, '.2f'))
21
22         # Read the next line.
23         line = sales_file.readline()
24
25     # Close the file.
26     sales_file.close()
27
28 # Call the main function.
```

```
29  main()
```

Program Output

```
 1000.00
2000.00
3000.00
4000.00
5000.00
```

Using Python's `for` Loop to Read Lines

In the previous example, you saw how the `readline` method returns an empty string when the end of the file has been reached. Most programming languages provide a similar technique for detecting the end of a file. If you plan to learn programming languages other than Python, it is important for you to know how to construct this type of logic.

The Python language also allows you to write a `for` loop that automatically reads the lines in a file without testing for any special condition that signals the end of the file. The loop does not require a

priming read operation, and it automatically stops when the end of the file has been reached. When you simply want to read the lines in a file, one after the other, this technique is simpler and more elegant than writing a `while` loop that explicitly tests for an end of the file condition. Here is the general format of the loop:

```
for variable in file_object:
    statement
    statement
    etc.
```

In the general format, *`variable`* is the name of a variable, and *`file_object`* is a variable that references a file object. The loop will iterate once for each line in the file. The first time the loop iterates, *`variable`* will reference the first line in the file (as a string), the second time the loop iterates, *`variable`* will reference the second line, and so forth. **Program 6-10** provides a demonstration. It reads and displays all of the items in the `sales.txt` file.

Program 6-10 *(read_sales2.py)*

```
1  # This program uses the for loop to read
2  # all of the values in the sales.txt file.
3
4  def main():
5      # Open the sales.txt file for reading.
```

```
 6       sales_file = open('sales.txt', 'r')
 7
 8       # Read all the lines from the file.
 9       for line in sales_file:
10           # Convert line to a float.
11           amount = float(line)
12           # Format and display the amount.
13           print(format(amount, '.2f'))
14
15       # Close the file.
16       sales_file.close()
17
18   # Call the main function.
19   main()
```

Program Output

```
1000.00
2000.00
3000.00
4000.00
5000.00
```

In the

Spotlight: Working with Files

Kevin is a freelance video producer who makes TV commercials for local businesses. When he makes a commercial, he usually films several short videos. Later, he puts these short videos together to make the final commercial. He has asked you to write the following two programs.

1. A program that allows him to enter the running time (in seconds) of each short video in a project. The running times are saved to a file.
2. A program that reads the contents of the file, displays the running times, and then displays the total running time of all the segments.

Here is the general algorithm for the first program, in pseudocode:

Get the number of videos in the project.

Open an output file.

For each video in the project:

    Get the video's running time.

    Write the running time to the file.

Close the file.

Program 6-11 shows the code for the first program.

Program 6-11

(save_running_times.py)

```
1  # This program saves a sequence of video running
times
 2  # to the video_times.txt file.
 3
 4  def main():
 5      # Get the number of videos in the project.
 6      num_videos = int(input('How many videos are in
the project? '))
 7
 8      # Open the file to hold the running times.
 9      video_file = open('video_times.txt', 'w')
10
11      # Get each video's running time and write
12      # it to the file.
13      print('Enter the running times for each
video.')
14      for count in range(1, num_videos + 1):
15          run_time = float(input('Video #' +
str(count) + ': '))
16          video_file.write(str(run_time) + '\n')
17
18      # Close the file.
19      video_file.close()
```

```
20      print('The times have been saved to
video_times.txt.')
21
22  # Call the main function.
23  main()
```

Program Output (with input shown in bold)

```
How many videos are in the project? 6 Enter
Enter the running times for each video.
Video #1: 24.5 Enter
Video #2: 12.2 Enter
Video #3: 14.6 Enter
Video #4: 20.4 Enter
Video #5: 22.5 Enter
Video #6: 19.3 Enter
The times have been saved to video_times.txt.
```

Here is the general algorithm for the second program:

Initialize an accumulator to 0.

Initialize a count variable to 0.

Open the input file.

For each line in the file:

> *Convert the line to a floating-point number. (This is the running time for a video.)*

Add one to the count variable. (This keeps count of the number of videos.)

Display the running time for this video.

Add the running time to the accumulator.

Close the file.

Display the contents of the accumulator as the total running time.

Program 6-12 shows the code for the second program.

Program 6-12

(read_running_times.py)

```
1  # This program the values in the video_times.txt
2  # file and calculates their total.
3
4  def main():
5      # Open the video_times.txt file for reading.
6      video_file = open('video_times.txt', 'r')
7
8      # Initialize an accumulator to 0.0.
9      total = 0.0
10
11     # Initialize a variable to keep count of the
```

```
videos.
    count = 0

    print('Here are the running times for each
video:')

    # Get the values from the file and total them.
    for line in video_file:
        # Convert a line to a float.
        run_time = float(line)

      # Add 1 to the count variable.
      count += 1

      # Display the time.
      print('Video #', count, ': ', run_time,
sep='')

      # Add the time to total.
      total += run_time

    # Close the file.
    video_file.close()

    # Display the total of the running times.
    print('The total running time is', total,
'seconds.')
```

```
36  # Call the main function.
37  main()
```

Program Output

```
 Here are the running times for each video:
Video #1: 24.5
Video #2: 12.2
Video #3: 14.6
Video #4: 20.4
Video #5: 22.5
Video #6: 19.3
The total running time is 113.5 seconds.
```

Checkpoint

6.12 Write a short program that uses a `for` loop to write the numbers 1 through 10 to a file.

6.13 What does it mean when the `readline` method returns an empty string?

6.14 Assume the file `data.txt` exists and contains several lines of text. Write a short program using the `while` loop that displays each line in the file.

6.15 Revise the program that you wrote for Checkpoint 6.14 to use the `for` loop instead of the `while` loop.

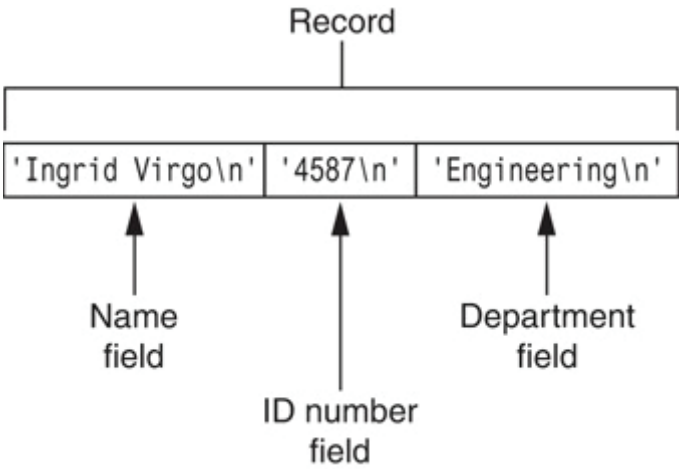
6.3 Processing Records

Concept:

The data that is stored in a file is frequently organized in records. A record is a complete set of data about an item, and a field is an individual piece of data within a record.

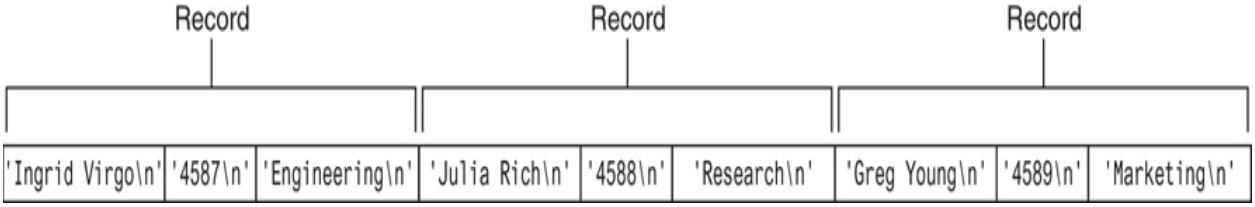
When data is written to a file, it is often organized into records and fields. A *record* is a complete set of data that describes one item, and a *field* is a single piece of data within a record. For example, suppose we want to store data about employees in a file. The file will contain a record for each employee. Each record will be a collection of fields, such as name, ID number, and department. This is illustrated in **Figure 6-18**.

Figure 6-18 Fields in a record



Each time you write a record to a sequential access file, you write the fields that make up the record, one after the other. For example, **Figure 6-19** shows a file that contains three employee records. Each record consists of the employee's name, ID number, and department.

Figure 6-19 Records in a file



Program 6-13 shows a simple example of how employee records can be written to a file.

Program 6-13 (save_emp_records.py)

```
1  # This program gets employee data from the user and
2  # saves it as records in the employee.txt file.
3
```

```
 4  def main():
 5      # Get the number of employee records to create.
 6      num_emps = int(input('How many employee records ' +
 7                           'do you want to create? '))
 8
 9      # Open a file for writing.
10      emp_file = open('employees.txt', 'w')
11
12      # Get each employee's data and write it to
13      # the file.
14      for count in range(1, num_emps + 1):
15          # Get the data for an employee.
16          print('Enter data for employee #', count, sep='')
17          name = input('Name: ')
18          id_num = input('ID number: ')
19          dept = input('Department: ')
20
21          # Write the data as a record to the file.
22          emp_file.write(name + '\n')
23          emp_file.write(id_num + '\n')
24          emp_file.write(dept + '\n')
25
26          # Display a blank line.
27          print()
28
29      # Close the file.
30      emp_file.close()
31      print('Employee records written to employees.txt.')
```

```
32
33  # Call the main function.
34  main()
```

Program Output (with input shown in bold)

```
How many employee records do you want to create? 3 [Enter]
Enter the data for employee #1

Name: Ingrid Virgo [Enter]
ID number: 4587 [Enter]
Department: Engineering [Enter]
Enter the data for employee #2

Name: Julia Rich [Enter]
ID number: 4588 [Enter]
Department: Research [Enter]
Enter the data for employee #3

Name: Greg Young [Enter]
ID number: 4589 [Enter]
Department: Marketing [Enter]
Employee records written to employees.txt.
```

The statement in lines 6 and 7 prompts the user for the number of employee records that he or she wants to create. Inside the loop, in lines 17 through 19, the program gets an employee's name, ID

number, and department. These three items, which together make an employee record, are written to the file in lines 22 through 24. The loop iterates once for each employee record.

When we read a record from a sequential access file, we read the data for each field, one after the other, until we have read the complete record. **Program 6-14** demonstrates how we can read the employee records in the `employee.txt` file.

Program 6-14 (read_emp_records.py)

```
 1  # This program displays the records that are
 2  # in the employees.txt file.
 3
 4  def main():
 5      # Open the employees.txt file.
 6      emp_file = open('employees.txt', 'r')
 7
 8      # Read the first line from the file, which is
 9      # the name field of the first record.
10      name = emp_file.readline()
11
12      # If a field was read, continue processing.
13      while name != '':
14          # Read the ID number field.
15          id_num = emp_file.readline()
16
17          # Read the department field.
```

```
18          dept = emp_file.readline()
19
20          # Strip the newlines from the fields.
21          name = name.rstrip('\n')
22          id_num = id_num.rstrip('\n')
23          dept = dept.rstrip('\n')
24
25          # Display the record.
26          print('Name:', name)
27          print('ID:', id_num)
28          print('Dept:', dept)
29          print()
30
31          # Read the name field of the next record.
32          name = emp_file.readline()
33
34      # Close the file.
35      emp_file.close()
36
37  # Call the main function.
38  main()
```

Program Output

```
Name: Ingrid Virgo
ID: 4587
Dept: Engineering
```

```
Name: Julia Rich
ID: 4588
Dept: Research

Name: Greg Young
ID: 4589
Dept: Marketing
```

This program opens the file in line 6, then in line 10 reads the first field of the first record. This will be the first employee's name. The `while` loop in line 13 tests the value to determine whether it is an empty string. If it is not, then the loop iterates. Inside the loop, the program reads the record's second and third fields (the employee's ID number and department), and displays them. Then, in line 32 the first field of the next record (the next employee's name) is read. The loop starts over and this process continues until there are no more records to read.

Programs that store records in a file typically require more capabilities than simply writing and reading records. In the following In the Spotlight sections, we will examine algorithms for adding records to a file, searching a file for specific records, modifying a record, and deleting a record.

In the

Spotlight: Adding and Displaying Records

Midnight Coffee Roasters, Inc. is a small company that imports raw coffee beans from around the world and roasts them to create a variety of gourmet coffees. Julie, the owner of the company, has asked you to write a series of programs that she can use to manage her inventory. After speaking with her, you have determined that a file is needed to keep inventory records. Each record should have two fields to hold the following data:

- Description. A string containing the name of the coffee
- Quantity in inventory. The number of pounds in inventory, as a floating-point number

Your first job is to write a program that can be used to add records to the file. **Program 6-15** shows the code. Note the output file is opened in append mode. Each time the program is executed, the new records will be added to the file's existing contents.

Program 6-15

`(add_coffee_record.py)`

```
1  # This program adds coffee inventory records to
```

```
 2  # the coffee.txt file.
 3
 4  def main():
 5      # Create a variable to control the loop.
 6      another = 'y'
 7
 8      # Open the coffee.txt file in append mode.
 9      coffee_file = open('coffee.txt', 'a')
10
11      # Add records to the file.
12      while another == 'y' or another == 'Y':
13          # Get the coffee record data.
14          print('Enter the following coffee data:')
15          descr = input('Description: ')
16          qty = int(input('Quantity (in pounds): '))
17
18          # Append the data to the file.
19          coffee_file.write(descr + '\n')
20          coffee_file.write(str(qty) + '\n')
21
22          # Determine whether the user wants to add
23          # another record to the file.
24          print('Do you want to add another record?')
25          another = input('Y = yes, anything else =
no: ')
26
27      # Close the file.
28      coffee_file.close()
```

```
29      print('Data appended to coffee.txt.')
30
31  # Call the main function.
32  main()
```

Program Output (with input shown in bold)

```
Enter the following coffee data:
Description: Brazilian Dark Roast Enter
Quantity (in pounds): 18 Enter
Do you want to enter another record?
Y = yes, anything else = no: y Enter
Description: Sumatra Medium Roast Enter
Quantity (in pounds): 25 Enter
Do you want to enter another record?
Y = yes, anything else = no: n Enter
Data appended to coffee.txt.
```

Your next job is to write a program that displays all of the records in the inventory file. **Program 6-16** shows the code.

Program 6-16

(show_coffee_records.py)

```
1  # This program displays the records in the
```

```
 2  # coffee.txt file.
 3
 4  def main():
 5      # Open the coffee.txt file.
 6      coffee_file = open('coffee.txt', 'r')
 7
 8      # Read the first record's description field.
 9      descr = coffee_file.readline()
10
11      # Read the rest of the file.
12      while descr != '':
13          # Read the quantity field.
14          qty = float(coffee_file.readline())
15
16          # Strip the \n from the description.
17          descr = descr.rstrip('\n')
18
19          # Display the record.
20          print('Description:', descr)
21          print('Quantity:', qty)
22
23          # Read the next description.
24          descr = coffee_file.readline()
25
26      # Close the file.
27      coffee_file.close()
28
29  # Call the main function.
```

```
30  main()
```

Program Output

```
Description: Brazilian Dark Roast
Quantity: 18.0
Description: Sumatra Medium Roast
Quantity: 25.0
```

In the Spotlight: Searching for a Record

Julie has been using the first two programs that you wrote for her. She now has several records stored in the `coffee.txt` file and has asked you to write another program that she can use to search for records. She wants to be able to enter a description and see a list of all the records matching that description. **Program 6-17** shows the code for the program.

Program 6-17

`(search_coffee_records.py)`

```
 1  # This program allows the user to search the
 2  # coffee.txt file for records matching a
```

```
 3  # description.
 4
 5  def main():
 6      # Create a bool variable to use as a flag.
 7      found = False
 8
 9      # Get the search value.
10      search = input('Enter a description to search
for: ')
11
12      # Open the coffee.txt file.
13      coffee_file = open('coffee.txt', 'r')
14
15      # Read the first record's description field.
16      descr = coffee_file.readline()
17
18      # Read the rest of the file.
19      while descr != '':
20          # Read the quantity field.
21          qty = float(coffee_file.readline())
22
23          # Strip the \n from the description.
24          descr = descr.rstrip('\n')
25
26          # Determine whether this record matches
27          # the search value.
28          if descr == search:
29              # Display the record.
```

```
30                print('Description:', descr)
31                print('Quantity:', qty)
32                print()
33                # Set the found flag to True.
34                found = True
35
36            # Read the next description.
37            descr = coffee_file.readline()
38
39        # Close the file.
40        coffee_file.close()
41
42        # If the search value was not found in the file
43        # display a message.
44        if not found:
45            print('That item was not found in the
file.')
46
47  # Call the main function.
48  main()
```

Program Output (with input shown in bold)

```
Enter a description to search for: Sumatra Medium Roast
Enter
Description: Sumatra Medium Roast
Quantity: 25.0
```

Program Output (with input shown in bold)

```
Enter a description to search for: Mexican Altura Enter
That item was not found in the file.
```

In the Spotlight: Modifying Records

Julie is very happy with the programs that you have written so far. Your next job is to write a program that she can use to modify the quantity field in an existing record. This will allow her to keep the records up to date as coffee is sold or more coffee of an existing type is added to inventory.

To modify a record in a sequential file, you must create a second temporary file. You copy all of the original file's records to the temporary file, but when you get to the record that is to be modified, you do not write its old contents to the temporary file. Instead, you write its new modified values to the temporary file. Then, you finish copying any remaining records from the original file to the temporary file.

The temporary file then takes the place of the original file. You delete the original file and rename the temporary file, giving it the name that the original file had on the computer's disk. Here is the general algorithm for your program.

> *Open the original file for input and create a temporary file for output.*
>
> *Get the description of the record to be modified and the new value for the quantity.*
>
> *Read the first description field from the original file.*
>
> *While the description field is not empty:*
>
> > *Read the quantity field.*
> >
> > *If this record's description field matches the description entered:*
> >
> > > *Write the new data to the temporary file.*
> >
> > *Else:*
> >
> > > *Write the existing record to the temporary file.*
> >
> > *Read the next description field.*
>
> *Close the original file and the temporary file.*
>
> *Delete the original file.*
>
> *Rename the temporary file, giving it the name of the original file.*

Notice at the end of the algorithm you delete the original file then rename the temporary file. The Python standard library's `os` module provides a function named `remove`, that deletes a file on the disk. You simply pass the name of the file as an

argument to the function. Here is an example of how you would delete a file named `coffee.txt`:

```
remove('coffee.txt')
```

The `os` module also provides a function named rename, that renames a file. Here is an example of how you would use it to rename the file `temp.txt` to `coffee.txt`:

```
rename('temp.txt', 'coffee.txt')
```

Program 6-18 shows the code for the program.

Program 6-18

(modify_coffee_records.py)

```
 1  # This program allows the user to modify the
quantity
 2  # in a record in the coffee.txt file.
 3
 4  import os # Needed for the remove and rename
functions
 5
 6  def main():
```

```
 7       # Create a bool variable to use as a flag.
 8       found = False
 9
10       # Get the search value and the new quantity.
11       search = input('Enter a description to search
for: ')
12       new_qty = int(input('Enter the new quantity:
'))
13
14       # Open the original coffee.txt file.
15       coffee_file = open('coffee.txt', 'r')
16
17       # Open the temporary file.
18       temp_file = open('temp.txt', 'w')
19
20       # Read the first record's description field.
21       descr = coffee_file.readline()
22
23       # Read the rest of the file.
24       while descr != '':
25           # Read the quantity field.
26           qty = float(coffee_file.readline())
27
28           # Strip the \n from the description.
29           descr = descr.rstrip('\n')
30
31           # Write either this record to the temporary
file,
```

```
            # or the new record if this is the one that
is
            # to be modified.
            if descr == search:
                # Write the modified record to the temp
file.
                temp_file.write(descr + '\n')
                temp_file.write(str(new_qty) + '\n')

                # Set the found flag to True.
                found = True
            else:
                # Write the original record to the temp
file.
                temp_file.write(descr + '\n')
                temp_file.write(str(qty) + '\n')

            # Read the next description.
            descr = coffee_file.readline()

        # Close the coffee file and the temporary file.
        coffee_file.close()
        temp_file.close()

        # Delete the original coffee.txt file.
        os.remove('coffee.txt')

        # Rename the temporary file.
```

```
57      os.rename('temp.txt', 'coffee.txt')
58
59      # If the search value was not found in the file
60      # display a message.
61      if found:
62          print('The file has been updated.')
63      else:
64          print('That item was not found in the
file.')
65
66  # Call the main function.
67  main()
```

Program Output (with input shown in bold)

```
Enter a description to search for: Brazilian Dark Roast
Enter
Enter the new quantity: 10 Enter
The file has been updated.
```

Note:

When working with a sequential access file, it is necessary to copy the entire file each time one item in the file is modified. As you can imagine, this approach is inefficient, especially if the

file is large. Other, more advanced techniques are available, especially when working with direct access files, that are much more efficient. We do not cover those advanced techniques in this book, but you will probably study them in later courses.

In the Spotlight: Deleting Records

Your last task is to write a program that Julie can use to delete records from the `coffee.txt` file. Like the process of modifying a record, the process of deleting a record from a sequential access file requires that you create a second temporary file. You copy all of the original file's records to the temporary file, except for the record that is to be deleted. The temporary file then takes the place of the original file. You delete the original file and rename the temporary file, giving it the name that the original file had on the computer's disk. Here is the general algorithm for your program.

Open the original file for input and create a temporary file for output.

Get the description of the record to be deleted.

Read the description field of the first record in the original file.

While the description is not empty:

Read the quantity field.

If this record's description field does not match the description entered:

Write the record to the temporary file.

Read the next description field.

Close the original file and the temporary file.

Delete the original file.

Rename the temporary file, giving it the name of the original file.

Program 6-19 shows the program's code.

Program 6-19

(delete_coffee_record.py)

```
 1  # This program allows the user to delete
 2  # a record in the coffee.txt file.
 3
 4  import os # Needed for the remove and rename
functions
 5
 6  def main():
 7      # Create a bool variable to use as a flag.
```

```
 8       found = False
 9
10       # Get the coffee to delete.
11       search = input('Which coffee do you want to
delete? ')
12
13       # Open the original coffee.txt file.
14       coffee_file = open('coffee.txt', 'r')
15
16       # Open the temporary file.
17       temp_file = open('temp.txt', 'w')
18
19       # Read the first record's description field.
20       descr = coffee_file.readline()
21
22       # Read the rest of the file.
23       while descr != '':
24           # Read the quantity field.
25           qty = float(coffee_file.readline())
26
27           # Strip the \n from the description.
28           descr = descr.rstrip('\n')
29
30           # If this is not the record to delete, then
31           # write it to the temporary file.
32           if descr != search:
33               # Write the record to the temp file.
34               temp_file.write(descr + '\n')
```

```
35                temp_file.write(str(qty) + '\n')
36            else:
37                # Set the found flag to True.
38                found = True
39
40            # Read the next description.
41            descr = coffee_file.readline()
42
43        # Close the coffee file and the temporary file.
44        coffee_file.close()
45        temp_file.close()
46
47        # Delete the original coffee.txt file.
48        os.remove('coffee.txt')
49
50        # Rename the temporary file.
51        os.rename('temp.txt', 'coffee.txt')
52
53        # If the search value was not found in the file
54        # display a message.
55        if found:
56            print('The file has been updated.')
57        else:
58            print('That item was not found in the
file.')
59
60  # Call the main function.
61  main()
```

Program Output (with input shown in bold)

```
Which coffee do you want to delete? Brazilian Dark
Roast Enter
The file has been updated.
```

Note:

When working with a sequential access file, it is necessary to copy the entire file each time one item in the file is deleted. As was previously mentioned, this approach is inefficient, especially if the file is large. Other, more advanced techniques are available, especially when working with direct access files, that are much more efficient. We do not cover those advanced techniques in this book, but you will probably study them in later courses.

Checkpoint

6.16 What is a record? What is a field?

6.17 Describe the way that you use a temporary file in a program that modifies a record in a sequential access file.
6.18 Describe the way that you use a temporary file in a program that deletes a record from a sequential file.

6.4 Exceptions

Concept:

An exception is an error that occurs while a program is running, causing the program to abruptly halt. You can use the try/except statement to gracefully handle exceptions.

An exception is an error that occurs while a program is running. In most cases, an exception causes a program to abruptly halt. For example, look at **Program 6-20**. This program gets two numbers from the user then divides the first number by the second number. In the sample running of the program, however, an exception occurred because the user entered 0 as the second number. (Division by 0 causes an exception because it is mathematically impossible.)

Program 6-20 `(division.py)`

```
1  # This program divides a number by another number.
2
3  def main():
4      # Get two numbers.
```

```
 5       num1 = int(input('Enter a number: '))
 6       num2 = int(input('Enter another number: '))
 7
 8       # Divide num1 by num2 and display the result.
 9       result = num1 / num2
10       print(num1, 'divided by', num2, 'is', result)
11
12   # Call the main function.
13   main()
```

Program Output (with input shown in bold)

```
Enter a number: 10 Enter
Enter another number: 0 Enter
Traceback (most recent call last):
  File "C:\Python\division.py," line 13, in <module>
      main()
  File "C:\Python\division.py," line 9, in main
      result = num1 / num2
ZeroDivisionError: integer division or modulo by zero
```

The lengthy error message that is shown in the sample run is called a *traceback*. The traceback gives information regarding the line number(s) that caused the exception. (When an exception occurs, programmers say that an exception was raised.) The last line of the error message shows the name of the exception that was raised

(`ZeroDivisionError`) and a brief description of the error that caused the exception to be raised (`integer division or modulo by zero`).

You can prevent many exceptions from being raised by carefully coding your program. For example, **Program 6-21** shows how division by 0 can be prevented with a simple `if` statement. Rather than allowing the exception to be raised, the program tests the value of `num2`, and displays an error message if the value is 0. This is an example of gracefully avoiding an exception.

Program 6-21 (division.py)

```
 1  # This program divides a number by another number.
 2
 3  def main():
 4      # Get two numbers.
 5      num1 = int(input('Enter a number: '))
 6      num2 = int(input('Enter another number: '))
 7
 8      # If num2 is not 0, divide num1 by num2
 9      # and display the result.
10      if num2 != 0:
11          result = num1 / num2
12          print(num1, 'divided by', num2, 'is', result)
13      else:
14          print('Cannot divide by zero.')
15
```

```
16  # Call the main function.
17  main()
```

Program Output (with input shown in bold)

```
Enter a number: 10 Enter
Enter another number: 0 Enter
Cannot divide by zero.
```

Some exceptions cannot be avoided regardless of how carefully you write your program. For example, look at **Program 6-22**. This program calculates gross pay. It prompts the user to enter the number of hours worked and the hourly pay rate. It gets the user's gross pay by multiplying these two numbers and displays that value on the screen.

Program 6-22 (gross_pay1.py)

```
 1  # This program calculates gross pay.
 2
 3  def main():
 4      # Get the number of hours worked.
 5      hours = int(input('How many hours did you work? '))
 6
 7      # Get the hourly pay rate.
 8      pay_rate = float(input('Enter your hourly pay rate:
```

```
'))
 9
10      # Calculate the gross pay.
11      gross_pay = hours * pay_rate
12
13      # Display the gross pay.
14      print('Gross pay: $', format(gross_pay, ',.2f'),
sep='')
15
16  # Call the main function.
17  main()
```

Program Output (with input shown in bold)

```
How many hours did you work? forty Enter
Traceback (most recent call last):
  File "C:\Users\Tony\Documents\Python\Source
Code\Chapter 06\gross_pay1.py", line 17, in <module>
    main()
  File "C:\Users\Tony\Documents\Python\Source
Code\Chapter 06\gross_pay1.py", line 5, in main
    hours = int(input('How many hours did you work? '))
ValueError: invalid literal for int() with base 10: 'forty'
```

Look at the sample running of the program. An exception occurred because the user entered the string `'forty'` instead of the number 40

when prompted for the number of hours worked. Because the string `'forty'` cannot be converted to an integer, the `int()` function raised an exception in line 5, and the program halted. Look carefully at the last line of the traceback message, and you will see that the name of the exception is `ValueError`, and its description is: `invalid literal for int() with base 10: 'forty'`.

Python, like most modern programming languages, allows you to write code that responds to exceptions when they are raised, and prevents the program from abruptly crashing. Such code is called an *exception handler* and is written with the `try/except` statement. There are several ways to write a `try/except` statement, but the following general format shows the simplest variation:

```
try:
  statement
  statement
  etc.
except ExceptionName:
  statement
  statement
  etc.
```

First, the key word `try` appears, followed by a colon. Next, a code block appears which we will refer to as the *try suite*. The *try suite* is one or more statements that can potentially raise an exception.

After the try suite, an `except` *clause* appears. The `except` clause begins with the key word `except`, optionally followed by the name of an exception, and ending with a colon. Beginning on the next line is a block of statements that we will refer to as a *handler*.

When the `try/except` statement executes, the statements in the try suite begin to execute. The following describes what happens next:

- If a statement in the try suite raises an exception that is specified by the `ExceptionName` in an `except` clause, then the handler that immediately follows the `except` clause executes. Then, the program resumes execution with the statement immediately following the `try/except` statement.
- If a statement in the try suite raises an exception that is *not* specified by the `ExceptionName` in an `except` clause, then the program will halt with a traceback error message.
- If the statements in the try suite execute without raising an exception, then any `except` clauses and handlers in the statement are skipped, and the program resumes execution with the statement immediately following the `try/except` statement.

Program 6-23 shows how we can write a `try/except` statement to gracefully respond to a `ValueError` exception.

Program 6-23 `(gross_pay2.py)`

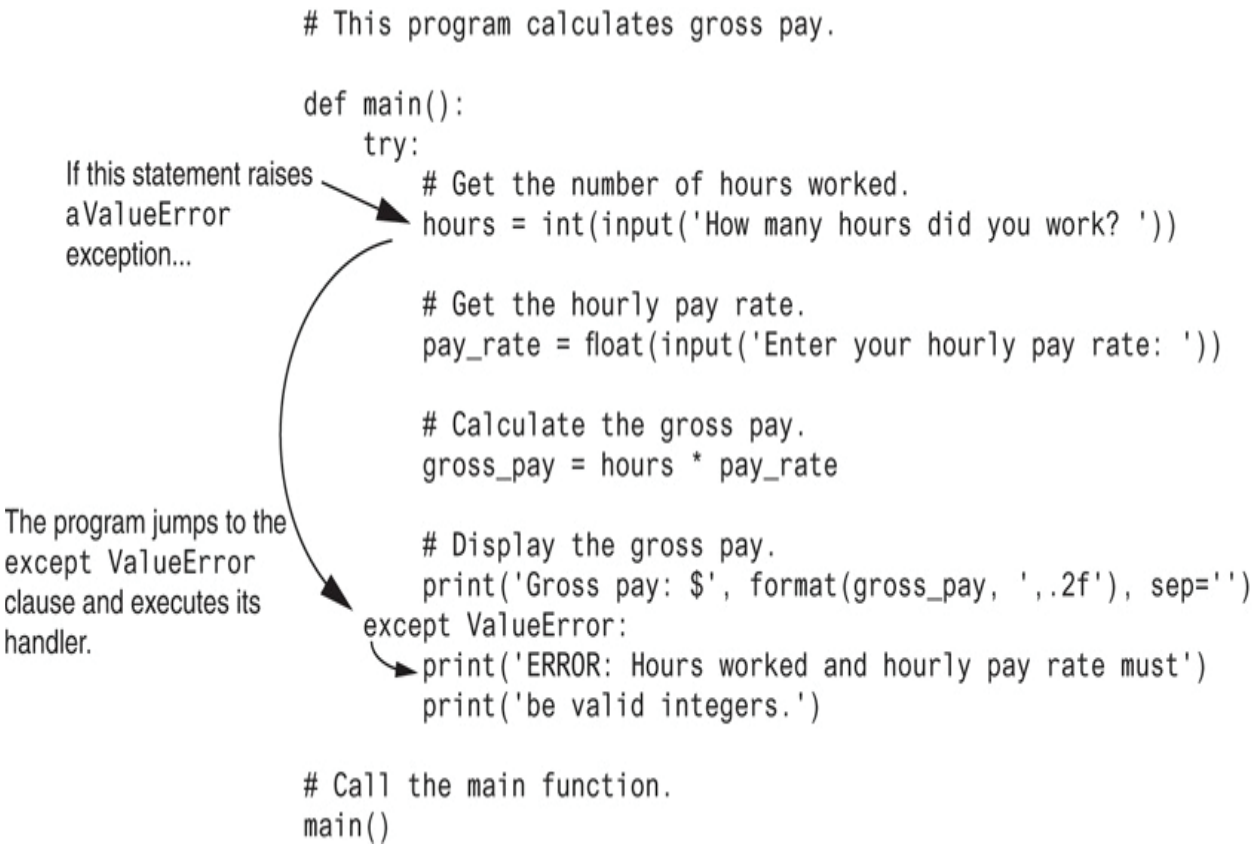
```
 1  # This program calculates gross pay.
 2
 3  def main():
 4      try:
 5          # Get the number of hours worked.
 6          hours = int(input('How many hours did you work?
'))
 7
 8          # Get the hourly pay rate.
 9          pay_rate = float(input('Enter your hourly pay
rate: '))
10
11          # Calculate the gross pay.
12          gross_pay = hours * pay_rate
13
14          # Display the gross pay.
15          print('Gross pay: $', format(gross_pay, ',.2f'),
sep='')
16      except ValueError:
17          print('ERROR: Hours worked and hourly pay rate
must')
18          print('be valid numbers.')
19
20  # Call the main function.
21  main()
```

Program Output (with input shown in bold)

```
How many hours did you work? forty Enter
ERROR: Hours worked and hourly pay rate must
be valid numbers.
```

Let’s look at what happened in the sample run. The statement in line 6 prompts the user to enter the number of hours worked, and the user enters the string `'forty'`. Because the string `'forty'` cannot be converted to an integer, the `int()` function raises a `ValueError` exception. As a result, the program jumps immediately out of the try suite to the `except ValueError` clause in line 16 and begins executing the handler block that begins in line 17. This is illustrated in **Figure 6-20**.

Figure 6-20 Handling an exception



Let's look at another example in **Program 6-24**. This program, which does not use exception handling, gets the name of a file from the user then displays the contents of the file. The program works as long as the user enters the name of an existing file. An exception will be raised, however, if the file specified by the user does not exist. This is what happened in the sample run.

Program 6-24 `(display_file.py)`

```
1  # This program displays the contents
2  # of a file.
3
4  def main():
```

```
    # Get the name of a file.
    filename = input('Enter a filename: ')

    # Open the file.
    infile = open(filename, 'r')

    # Read the file's contents.
    contents = infile.read()

    # Display the file's contents.
    print(contents)

    # Close the file.
    infile.close()

# Call the main function.
main()
```

Program Output (with input shown in bold)

```
Enter a filename: bad_file.txt Enter
Traceback (most recent call last):
File "C:\Python\display_file.py," line 21, in <module>
main()
File "C:\Python\display_file.py," line 9, in main
infile = open(filename, 'r')
IOError: [Errno 2] No such file or directory: 'bad_file.txt'
```

The statement in line 9 raised the exception when it called the `open` function. Notice in the traceback error message that the name of the exception that occurred is `IOError`. This is an exception that is raised when a file I/O operation fails. You can see in the traceback message that the cause of the error was `No such file or directory: 'bad_file.txt'`.

Program 6-25 shows how we can modify **Program 6-24** with a `try/except` statement that gracefully responds to an `IOError` exception. In the sample run, assume the file `bad_file.txt` does not exist.

Program 6-25 `(display_file2.py)`

```
 1  # This program displays the contents
 2  # of a file.
 3
 4  def main():
 5      # Get the name of a file.
 6      filename = input('Enter a filename: ')
 7
 8      try:
 9          # Open the file.
10          infile = open(filename, 'r')
11
12          # Read the file's contents.
```

```
13          contents = infile.read()
14
15          # Display the file's contents.
16          print(contents)
17
18          # Close the file.
19          infile.close()
20      except IOError:
21          print('An error occurred trying to read')
22          print('the file', filename)
23
24  # Call the main function.
25  main()
```

Program Output (with input shown in bold)

```
Enter a filename: bad_file.txt Enter
An error occurred trying to read the file bad_file.txt
```

Let's look at what happened in the sample run. When line 6 executed, the user entered `bad_file.txt`, which was assigned to the `filename` variable. Inside the try suite, line 10 attempts to open the file `bad_file.txt`. Because this file does not exist, the statement raises an `IOError` exception. When this happens, the program exits the try suite, skipping lines 11 through 19. Because the `except` clause in line 20

specifies the `IOError` exception, the program jumps to the handler that begins in line 21.

Handling Multiple Exceptions

In many cases, the code in a try suite will be capable of throwing more than one type of exception. In such a case, you need to write an `except` clause for each type of exception that you want to handle. For example, **Program 6-26** reads the contents of a file named `sales_data.txt`. Each line in the file contains the sales amount for one month, and the file has several lines. Here are the contents of the file:

```
24987.62
26978.97
32589.45
31978.47
22781.76
29871.44
```

Program 6-26 reads all of the numbers from the file and adds them to an accumulator variable.

Program 6-26 (sales_report1.py)

```
 1  # This program displays the total of the
 2  # amounts in the sales_data.txt file.
 3
 4  def main():
 5      # Initialize an accumulator.
 6      total = 0.0
 7
 8      try:
 9          # Open the sales_data.txt file.
10          infile = open('sales_data.txt', 'r')
11
12          # Read the values from the file and
13          # accumulate them.
14          for line in infile:
15              amount = float(line)
16              total += amount
17
18          # Close the file.
19          infile.close()
20
21          # Print the total.
22          print(format(total, ',.2f'))
23
24      except IOError:
25          print('An error occured trying to read the file.')
26
27      except ValueError:
28          print('Non-numeric data found in the file.')
```

```
29
30      except:
31          print('An error occured.')
32
33  # Call the main function.
34  main()
```

The try suite contains code that can raise different types of exceptions. For example:

- The statement in line 10 can raise an `IOError` exception if the `sales_data.txt` file does not exist. The `for` loop in line 14 can also raise an `IOError` exception if it encounters a problem reading data from the file.
- The `float` function in line 15 can raise a `ValueError` exception if the `line` variable references a string that cannot be converted to a floating-point number (an alphabetic string, for example).

Notice the `try/except` statement has three `except` clauses:

- The `except` clause in line 24 specifies the `IOError` exception. Its handler in line 25 will execute if an `IOError` exception is raised.
- The `except` clause in line 27 specifies the `ValueError` exception. Its handler in line 28 will execute if a `ValueError` exception is raised.
- The `except` clause in line 30 does not list a specific exception. Its handler in line 31 will execute if an exception that is not handled by

the other `except` clauses is raised.

If an exception occurs in the try suite, the Python interpreter examines each of the `except` clauses, from top to bottom, in the `try/except` statement. When it finds an `except` clause that specifies a type that matches the type of exception that occurred, it branches to that `except` clause. If none of the `except` clauses specifies a type that matches the exception, the interpreter branches to the `except` clause in line 30.

Using One `except` Clause to Catch All Exceptions

The previous example demonstrated how multiple types of exceptions can be handled individually in a `try/except` statement. Sometimes you might want to write a `try/except` statement that simply catches any exception that is raised in the try suite and, regardless of the exception's type, responds the same way. You can accomplish that in a `try/except` statement by writing one `except` clause that does not specify a particular type of exception. **Program 6-27** shows an example.

Program 6-27 *(sales_report2.py)*

```
1  # This program displays the total of the
```

```
# amounts in the sales_data.txt file.

def main():
    # Initialize an accumulator.
    total = 0.0

    try:
        # Open the sales_data.txt file.
        infile = open('sales_data.txt', 'r')

        # Read the values from the file and
        # accumulate them.
        for line in infile:
            amount = float(line)
            total += amount

        # Close the file.
        infile.close()

        # Print the total.
        print(format(total, ',.2f'))
    except:
        print('An error occurred.')

# Call the main function.
main()
```

Notice the `try/except` statement in this program has only one `except` clause, in line 23. The `except` clause does not specify an exception type, so any exception that occurs in the try suite (lines 9 through 22) causes the program to branch to line 23 and execute the statement in line 24.

Displaying an Exception's Default Error Message

When an exception is thrown, an object known as an *exception object* is created in memory. The exception object usually contains a default error message pertaining to the exception. (In fact, it is the same error message that you see displayed at the end of a traceback when an exception goes unhandled.) When you write an `except` clause, you can optionally assign the exception object to a variable, as shown here:

```
except ValueError as err:
```

This `except` clause catches `ValueError` exceptions. The expression that appears after the `except` clause specifies that we are assigning the exception object to the variable `err`. (There is nothing special about the name `err`. That is simply the name that we have chosen for

the examples. You can use any name that you wish.) After doing this, in the exception handler you can pass the `err` variable to the `print` function to display the default error message that Python provides for that type of error. **Program 6-28** shows an example of how this is done.

Program 6-28 *(gross_pay3.py)*

```
 1  # This program calculates gross pay.
 2
 3  def main():
 4      try:
 5          # Get the number of hours worked.
 6          hours = int(input('How many hours did you work?
'))
 7
 8          # Get the hourly pay rate.
 9          pay_rate = float(input('Enter your hourly pay
rate: '))
10
11          # Calculate the gross pay.
12          gross_pay = hours * pay_rate
13
14          # Display the gross pay.
15          print('Gross pay: $', format(gross_pay, ',.2f'),
sep='')
16      except ValueError as err:
17          print(err)
```

```
18
19  # Call the main function.
20  main()
```

Program Output (with input shown in bold)

```
How many hours did you work? forty
invalid literal for int() with base 10: 'forty'
```

When a `ValueError` exception occurs inside the try suite (lines 5 through 15), the program branches to the `except` clause in line 16. The expression `ValueError as err` in line 16 causes the resulting exception object to be assigned to a variable named `err`. The statement in line 17 passes the `err` variable to the `print` function, which causes the exception's default error message to be displayed.

If you want to have just one `except` clause to catch all the exceptions that are raised in a try suite, you can specify `Exception` as the type. **Program 6-29** shows an example.

Program 6-29 *(sales_report3.py)*

```
 1  # This program displays the total of the
 2  # amounts in the sales_data.txt file.
 3
```

```
 4  def main():
 5      # Initialize an accumulator.
 6      total = 0.0
 7
 8      try:
 9          # Open the sales_data.txt file.
10          infile = open('sales_data.txt', 'r')
11
12          # Read the values from the file and
13          # accumulate them.
14          for line in infile:
15              amount = float(line)
16              total += amount
17
18          # Close the file.
19          infile.close()
20
21          # Print the total.
22          print(format(total, ',.2f'))
23      except Exception as err:
24          print(err)
25
26  # Call the main function.
27  main()
```

The `else` Clause

The `try/except` statement may have an optional `else` clause, which appears after all the `except` clauses. Here is the general format of a `try/except` statement with an `else` clause:

```
try:
   statement
   statement
   etc.
except ExceptionName:
   statement
   statement
   etc.
else:
   statement
   statement
   etc.
```

The block of statements that appears after the `else` clause is known as the *else suite*. The statements in the else suite are executed after the statements in the try suite, only if no exceptions were raised. If an exception is raised, the else suite is skipped. **Program 6-30** shows an example.

Program 6-30 *(sales_report4.py)*

```
 1  # This program displays the total of the
```

```
# amounts in the sales_data.txt file.

def main():
    # Initialize an accumulator.
    total = 0.0

    try:
        # Open the sales_data.txt file.
        infile = open('sales_data.txt', 'r')

        # Read the values from the file and
        # accumulate them.
        for line in infile:
            amount = float(line)
            total += amount

        # Close the file.
        infile.close()
    except Exception as err:
        print(err)
    else:
        # Print the total.
        print(format(total, ',.2f'))

# Call the main function.
main()
```

In **Program 6-30**, the statement in line 24 is executed only if the statements in the try suite (lines 9 through 19) execute without raising an exception.

The `finally` Clause

The `try/except` statement may have an optional `finally` clause, which must appear after all the `except` clauses. Here is the general format of a `try/except` statement with a `finally` clause:

```
try:
   statement
   statement
   etc.
except ExceptionName:
   statement
   statement
   etc.
finally:
   statement
   statement
   etc.
```

The block of statements that appears after the `finally` clause is known as the *finally suite*. The statements in the finally suite are

always executed after the try suite has executed, and after any exception handlers have executed. The statements in the finally suite execute whether an exception occurs or not. The purpose of the finally suite is to perform cleanup operations, such as closing files or other resources. Any code that is written in the finally suite will always execute, even if the try suite raises an exception.

What If an Exception Is Not Handled?

Unless an exception is handled, it will cause the program to halt. There are two possible ways for a thrown exception to go unhandled. The first possibility is for the `try/except` statement to contain no `except` clauses specifying an exception of the right type. The second possibility is for the exception to be raised from outside a try suite. In either case, the exception will cause the program to halt.

In this section, you've seen examples of programs that can raise `ZeroDivisionError` exceptions, `IOError` exceptions, and `ValueError` exceptions. There are many different types of exceptions that can occur in a Python program. When you are designing `try/except` statements, one way you can learn about the exceptions that you need to handle is to consult the Python documentation. It gives detailed information about each possible exception and the types of errors that can cause them to occur.

Another way that you can learn about the exceptions that can occur in a program is through experimentation. You can run a program and deliberately perform actions that will cause errors. By watching the traceback error messages that are displayed, you will see the names of the exceptions that are raised. You can then write `except` clauses to handle these exceptions.

Checkpoint

6.19 Briefly describe what an exception is.

6.20 If an exception is raised and the program does not handle it with a `try/except` statement, what happens?

6.21 What type of exception does a program raise when it tries to open a nonexistent file?

6.22 What type of exception does a program raise when it uses the `float` function to convert a non-numeric string to a number?

Review Questions

Multiple Choice

1. A file that data is written to is known as a(n)_____.
    a. input file
    b. output file
    c. sequential access file
    d. binary file

2. A file that data is read from is known as a(n)_____.
    a. input file
    b. output file
    c. sequential access file
    d. binary file

3. Before a file can be used by a program, it must be_____.
    a. formatted
    b. encrypted
    c. closed
    d. opened

4. When a program is finished using a file, it should do this.
    a. erase the file

b. open the file
c. close the file
d. encrypt the file

5. The contents of this type of file can be viewed in an editor such as Notepad.
    a. text file
    b. binary file
    c. English file
    d. human-readable file

6. This type of file contains data that has not been converted to text.
    a. text file
    b. binary file
    c. Unicode file
    d. symbolic file

7. When working with this type of file, you access its data from the beginning of the file to the end of the file.
    a. ordered access
    b. binary access
    c. direct access
    d. sequential access

8. When working with this type of file, you can jump directly to any piece of data in the file without reading the data that comes before it.

a. ordered access
b. binary access
c. direct access
d. sequential access

9. This is a small “holding section” in memory that many systems write data to before writing the data to a file.
    a. buffer
    b. variable
    c. virtual file
    d. temporary file

10. This marks the location of the next item that will be read from a file.
    a. input position
    b. delimiter
    c. pointer
    d. read position

11. When a file is opened in this mode, data will be written at the end of the file’s existing contents.
    a. output mode
    b. append mode
    c. backup mode
    d. read-only mode

12. This is a single piece of data within a record.
    a. field

b. variable
c. delimiter
d. subrecord

13. When an exception is generated, it is said to have been _____.
    a. built
    b. raised
    c. caught
    d. killed

14. This is a section of code that gracefully responds to exceptions.
    a. exception generator
    b. exception manipulator
    c. exception handler
    d. exception monitor

15. You write this statement to respond to exceptions.
    a. `run/handle`
    b. `try/except`
    c. `try/handle`
    d. `attempt/except`

True or False

1. When working with a sequential access file, you can jump directly to any piece of data in the file without reading the data that comes before it.
2. When you open a file that file already exists on the disk using the `'w'` mode, the contents of the existing file will be erased.
3. The process of opening a file is only necessary with input files. Output files are automatically opened when data is written to them.
4. When an input file is opened, its read position is initially set to the first item in the file.
5. When a file that already exists is opened in append mode, the file's existing contents are erased.
6. If you do not handle an exception, it is ignored by the Python interpreter, and the program continues to execute.
7. You can have more than one `except` clause in a `try/except` statement.
8. The else suite in a `try/except` statement executes only if a statement in the try suite raises an exception.
9. The finally suite in a `try/except` statement executes only if no exceptions are raised by statements in the try suite.

Short Answer

1. Describe the three steps that must be taken when a file is used by a program.
2. Why should a program close a file when it's finished using it?

3. What is a file's read position? Where is the read position when a file is first opened for reading?
4. If an existing file is opened in append mode, what happens to the file's existing contents?
5. If a file does not exist and a program attempts to open it in append mode, what happens?

Algorithm Workbench

1. Write a program that opens an output file with the filename `my_name.txt`, writes your name to the file, then closes the file.
2. Write a program that opens the `my_name.txt` file that was created by the program in problem 1, reads your name from the file, displays the name on the screen, then closes the file.
3. Write code that does the following: opens an output file with the filename `number_list.txt`, uses a loop to write the numbers 1 through 100 to the file, then closes the file.
4. Write code that does the following: opens the `number_list.txt` file that was created by the code you wrote in question 3, reads all of the numbers from the file and displays them, then closes the file.
5. Modify the code that you wrote in problem 4 so it adds all of the numbers read from the file and displays their total.
6. Write code that opens an output file with the filename `number_list.txt`, but does not erase the file's contents if it already exists.

7. A file exists on the disk named `students.txt`. The file contains several records, and each record contains two fields: (1) the student's name, and (2) the student's score for the final exam. Write code that deletes the record containing "John Perz" as the student name.
8. A file exists on the disk named `students.txt`. The file contains several records, and each record contains two fields: (1) the student's name, and (2) the student's score for the final exam. Write code that changes Julie Milan's score to 100.
9. What will the following code display?

```
try:
  x = float('abc123')
  print('The conversion is complete.')
except IOError:
  print('This code caused an IOError.')
except ValueError:
  print('This code caused a ValueError.')
print('The end.')
```

10. What will the following code display?

```
try:
  x = float('abc123')
  print(x)
except IOError:
  print('This code caused an IOError.')
except ZeroDivisionError:
```

```
  print('This code caused a ZeroDivisionError.')
except:
  print('An error happened.')
print('The end.')
```

Programming Exercises

1. **File Display**
   Assume a file containing a series of integers is named `numbers.txt` and exists on the computer's disk. Write a program that displays all of the numbers in the file.

File Display

2. **File Head Display**
   Write a program that asks the user for the name of a file. The program should display only the first five lines of the file's contents. If the file contains less than five lines, it should display the file's entire contents.
3. **Line Numbers**
   Write a program that asks the user for the name of a file. The program should display the contents of the file with each line preceded with a line number followed by a colon. The line numbering should start at 1.

4. **Item Counter**
   Assume a file containing a series of names (as strings) is named `names.txt` and exists on the computer's disk. Write a program that displays the number of names that are stored in the file.
   (Hint: Open the file and read every string stored in it. Use a variable to keep a count of the number of items that are read from the file.)
5. **Sum of Numbers**
   Assume a file containing a series of integers is named `numbers.txt` and exists on the computer's disk. Write a program that reads all of the numbers stored in the file and calculates their total.
6. **Average of Numbers**
   Assume a file containing a series of integers is named `numbers.txt` and exists on the computer's disk. Write a program that calculates the average of all the numbers stored in the file.
7. **Random Number File Writer**
   Write a program that writes a series of random numbers to a file. Each random number should be in the range of 1 through 500. The application should let the user specify how many random numbers the file will hold.
8. **Random Number File Reader**
   This exercise assumes you have completed Programming Exercise 7, *Random Number File Writer*. Write another program that reads the random numbers from the file, displays the numbers, then displays the following data:
   - The total of the numbers

- The number of random numbers read from the file

9. **Exception Handing**
   Modify the program that you wrote for Exercise 6 so it handles the following exceptions:
   - It should handle any `IOError` exceptions that are raised when the file is opened and data is read from it.
   - It should handle any `ValueError` exceptions that are raised when the items that are read from the file are converted to a number.

10. **Golf Scores**
    The Springfork Amateur Golf Club has a tournament every weekend. The club president has asked you to write two programs:
    1. A program that will read each player's name and golf score as keyboard input, then save these as records in a file named `golf.txt`. (Each record will have a field for the player's name and a field for the player's score.)
    2. A program that reads the records from the `golf.txt` file and displays them.

11. **Personal Web Page Generator**
    Write a program that asks the user for his or her name, then asks the user to enter a sentence that describes himself or herself. Here is an example of the program's screen:

```
Enter your name: Julie Taylor [Enter]
```

```
Describe yourself: I am a computer science major, a member
of the
Jazz club, and I hope to work as a mobile app developer
after I
graduate.
```

Once the user has entered the requested input, the program should create an HTML file, containing the input, for a simple Web page. Here is an example of the HTML content, using the sample input previously shown:

```
<html>
<head>
</head>
<body>
   <center>
      <h1>Julie Taylor</h1>
   </center>
   <hr />
   I am a computer science major, a member of the Jazz
club,
   and I hope to work as a mobile app developer after I
graduate.
   <hr />
</body>
</html>
```

12. **Average Steps Taken**

A Personal Fitness Tracker is a wearable device that tracks your physical activity, calories burned, heart rate, sleeping patterns, and so on. One common physical activity that most of these devices track is the number of steps you take each day. If you have downloaded this book's source code from the Computer Science Portal, you will find a file named steps.txt in the **Chapter 06** folder. (The Computer Science Portal can be found at **www.pearsonhighered.com/gaddis**.) The steps.txt file contains the number of steps a person has taken each day for a year. There are 365 lines in the file, and each line contains the number of steps taken during a day. (The first line is the number of steps taken on January 1st, the second line is the number of steps taken on January 2nd, and so forth.) Write a program that reads the file, then displays the average number of steps taken for each month. (The data is from a year that was not a leap year, so February has 28 days.)

Chapter 7 Lists and Tuples

Topics